

SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.(BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 3

Duration : 1 Hour

Total Marks:30

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I ___A Bharath / 192512384 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A . Bharath

Name : A . Bharath

Reg No : 192512384

Course : CSA1718 -Artificial Intelligence

Question 1: Doctor Shift Assignment Using Backtracking Search

Objective

- To apply the Backtracking Search Algorithm to assign doctors to shifts while satisfying all given constraints and obtaining a valid schedule.

Problem Statement

A hospital needs to assign **3 doctors (D1, D2, D3)** to **3 shifts (Morning, Afternoon, Night)** while satisfying the following constraints:

Constraints

1. **D1 cannot work Night shift.**
 2. **D2 must work before D3** (Morning < Afternoon < Night).
 3. **Only one doctor can be assigned to each shift.**
 4. **D3 cannot work Morning shift.**
 5. **Every doctor must be assigned exactly one shift.**
-

Step 1: Variables and Domain

Variables

- D1
- D2
- D3

Domain (Possible Values)

- Morning (M)
 - Afternoon (A)
 - Night (N)
-

Step 2: Initial State

No assignments have been made.

Doctor Shift

D1 —

D2 —

D3 —

Step 3: Apply Backtracking Search

Backtracking assigns one doctor at a time. If a constraint is violated, it backtracks and tries another assignment.

Step 3.1: Assign D1

Possible shifts for D1:

- Morning ✓
- Afternoon ✓
- Night ✗ (Constraint 1)

Choose the first valid value:

D1 = Morning

Current Assignment:

Doctor Shift

D1 Morning

D2 —

D3 —

Constraint Check

- D1 cannot work Night ✓
 - Other constraints are not affected yet ✓
-

Step 3.2: Assign D2 Remaining

shifts:

- Afternoon
- Night

Try the first available value:

D2 = Afternoon

Current Assignment:

Doctor Shift

D1 Morning

D2 Afternoon

D3 —

Constraint Check

- One doctor per shift ✓
 - D2 before D3 cannot be checked until D3 is assigned ✓
-

Step 3.3: Assign D3

Only remaining shift:

D3 = Night

Current Assignment:

Doctor Shift

D1 Morning

D2 Afternoon

D3 Night

Constraint Check

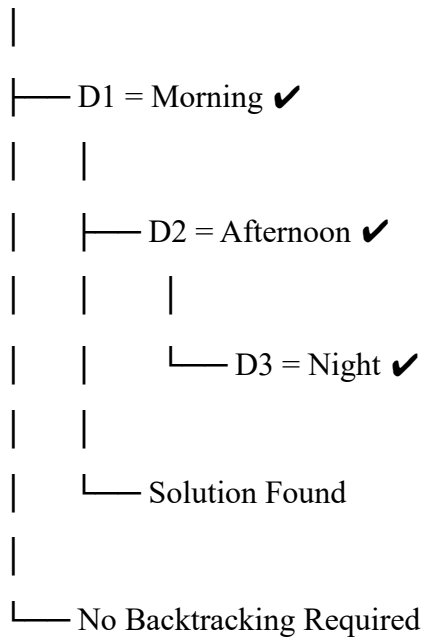
1. D3 cannot work Morning ✓ (Assigned Night)
2. D2 must work before D3 ✓

- Afternoon comes before Night.
3. Only one doctor per shift ✓
 4. Every doctor is assigned exactly one shift ✓

All constraints are satisfied.

Backtracking Search Tree

Start



Constraint Verification Table

Constraint	Verification	Status
D1 cannot work Night	D1 = Morning	✓ Satisfied
D2 must work before D3	Afternoon < Night	✓ Satisfied
One doctor per shift	Each shift has one doctor	✓ Satisfied
D3 cannot work Morning	D3 = Night	✓ Satisfied
Every doctor assigned exactly one shift	Yes	✓ Satisfied

Final Valid Schedule

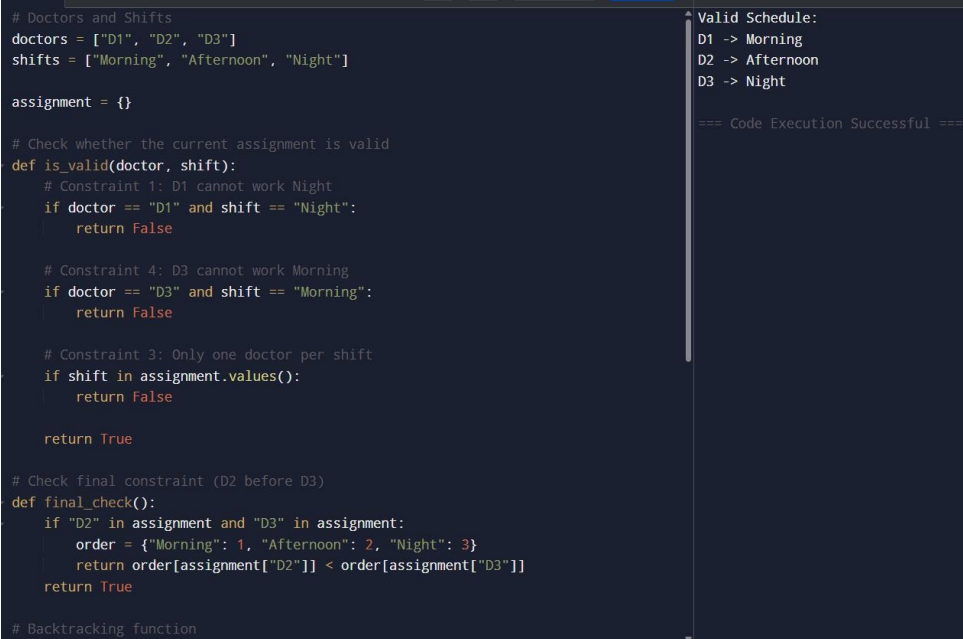
Doctor Assigned Shift

D1 Morning

D2 Afternoon

D3 Night

Out Put



```
# Doctors and Shifts
doctors = ["D1", "D2", "D3"]
shifts = ["Morning", "Afternoon", "Night"]

assignment = {}

# Check whether the current assignment is valid
def is_valid(doctor, shift):
    # Constraint 1: D1 cannot work Night
    if doctor == "D1" and shift == "Night":
        return False

    # Constraint 4: D3 cannot work Morning
    if doctor == "D3" and shift == "Morning":
        return False

    # Constraint 3: Only one doctor per shift
    if shift in assignment.values():
        return False

    return True

# Check final constraint (D2 before D3)
def final_check():
    if "D2" in assignment and "D3" in assignment:
        order = {"Morning": 1, "Afternoon": 2, "Night": 3}
        return order[assignment["D2"]] < order[assignment["D3"]]
    return True

# Backtracking function
```

Valid Schedule:
D1 -> Morning
D2 -> Afternoon
D3 -> Night

=== Code Execution Successful ===

Conclusion

Using the **Backtracking Search algorithm**, the assignments were made one by one while checking all constraints at each step. Since no constraint was violated, backtracking was not required. The final valid schedule is:

- **D1 → Morning**
- **D2 → Afternoon**

Question 2: BFS Search Algorithm (Short Answer)

Objective

- To implement the Breadth-First Search (BFS) Algorithm to find the shortest path from the start node to the goal node in a grid while avoiding obstacles and minimizing the path cost.

Given Constraints

- Grid size: 5×5
- Start node: **S**
- Goal node: **G**
- Allowed moves: **Up, Down, Left, Right**
- Cost of each move = **1** • Obstacles cannot be crossed.
- Manhattan Distance heuristic:

BFS Steps

$$h(n) = |x_{goal} - x| + |y_{goal} - y|$$

1. Start from node **S**.
2. Add the start node to the queue.
3. Remove the first node from the queue and expand it.
4. Add all valid neighboring nodes to the queue.
5. Mark visited nodes to avoid revisiting.
6. Continue until the goal node **G** is reached.

Optimal Path

$S \rightarrow (0,1) \rightarrow (0,2) \rightarrow (1,2) \rightarrow (2,2) \rightarrow (2,3) \rightarrow (2,4) \rightarrow (3,4) \rightarrow G$

Total Cost

- Number of moves = **8**
- Cost per move = **1**
- **Total Cost = 8**

Out Put

```

from collections import deque

# 5x5 Grid
grid = [
    ['S', ' ', ' ', 'X', ' '],
    [' ', 'X', ' ', 'X', ' '],
    [' ', 'X', ' ', ' ', ' '],
    [' ', ' ', 'X', 'X', ' '],
    ['X', ' ', ' ', ' ', 'G']
]

rows = len(grid)
cols = len(grid[0])

# Start and Goal positions
start = (0, 0)
goal = (4, 4)

# Possible movements: Down, Up, Right, Left
directions = [
    (1, 0),
    (-1, 0),
    (0, 1),
    (0, -1)
]

# BFS Function
def bfs():
    queue = deque([start])
    visited = set([start])

    Queue: [(2, 4), (4, 3)]

    Step 13
    Expanded Node: (2, 4)
    Queue: [(4, 3), (3, 4), (1, 4)]

    Step 14
    Expanded Node: (4, 3)
    Queue: [(3, 4), (1, 4), (4, 4)]

    Step 15
    Expanded Node: (3, 4)
    Queue: [(1, 4), (4, 4)]

    Step 16
    Expanded Node: (1, 4)
    Queue: [(4, 4), (0, 4)]

    Step 17
    Expanded Node: (4, 4)

    Goal Reached!

    Optimal Path:
    (0, 0) -> (1, 0) -> (2, 0) -> (3, 0) -> (3, 1) -> (4, 1) -> (4, 2) -> (4, 3) -> (4, 4)
    -> Goal

    Total Cost = 8

== Code Execution Successful ==

```

Conclusion

Using **Breadth-First Search (BFS)**, the robot finds the shortest path from **Start (S)** to **Goal (G)** with a minimum cost of **8**.

Question 3: Autonomous Rescue Robot (Short Answer)

Objective

- To analyze the decision-making process of an autonomous rescue robot and apply Uniform Cost Search (UCS) with an Online Search Agent to achieve safe, efficient, and minimum-cost navigation in a partially observable environment.

a) Constraint Analysis

- The robot can move only **Up, Down, Left, and Right**.
- Obstacles cannot be crossed.
- The robot can observe only adjacent cells.
- Each move costs **1**.
- Risky zones add an extra cost of **2**.
- The robot updates its knowledge while exploring.

b)

Solution Approach

Use **Uniform Cost Search (UCS)** with an **Online Search Agent**.

Steps:

1. Start from **S**.
2. Observe nearby cells.
3. Avoid obstacles.
4. Calculate the total path cost.
5. Choose the path with the lowest cost.
6. Update knowledge as new information is discovered.
7. Continue until reaching **G**.

c)

AI Concepts

- **Intelligent Agent:** Perceives the environment, makes decisions, and acts to reach the goal.
- **Rationality:** Chooses the safest and least-cost path.
- **Environment Type:** Partially observable, dynamic, sequential, deterministic, and discrete.

Justification

Uniform Cost Search (UCS) is the best choice because it:

- Finds the **minimum-cost path**.
- Considers both movement and risk costs.
- Avoids obstacles.
- Adapts to newly discovered information during navigation.
- Ensures safe and efficient rescue.

Out Put

```

1 import heapq
2
3 # Grid
4 grid = [
5     ['S', '.', '.', '#', '.'],
6     ['.', '#', '.', '#', '.'],
7     ['.', 'R', '.', '.', '.'],
8     ['.', '.', '#', '#', '.'],
9     ['#', '.', '.', '.', 'G']
10 ]
11
12 rows = len(grid)
13 cols = len(grid[0])
14
15 start = (0, 0)
16 goal = (4, 4)
17
18 # Directions: Up, Down, Left, Right
19 directions = [(-1,0),(1,0),(0,-1),(0,1)]
20
21 # Uniform Cost Search
22 def ucs():
23     pq = [(0, start)]          # (cost, node)
24     visited = set()
25     parent = {}
26
27     while pq:
28         cost, current = heapq.heappop(pq)
29
30         if current in visited:

```

```

Goal Reached!
Minimum Cost = 8
Path:
(0, 0) -> (1, 0) -> (2, 0) -> (3, 0) -> (3, 1) -> (4, 1) -> (4, 2) -> (4, 3) -> (4, 4) ->
Goal

=== Code Execution Successful ===

```

Conclusion

The rescue robot uses **UCS with an Online Search Agent** to safely reach the survivor while minimizing the total travel cost.